

Learn Low-Level Design in 25 Days: Day 6—UML Basics Every Engineer Should Know

When Whiteboards Become Code

TechPreneur

Published October 18, 2025

Free: No

Read on Freedium · original

Contents

Why UML Exists 0

A. Class Diagrams — The Skeleton of Your System 0

B. Sequence Diagrams — The Flow of a Feature 0

UML in a Nutshell 0

Try It Yourself 0

Summary + Next-Day Teaser 0

Picture this: you're in a design meeting, markers in hand, trying to explain your new "Notification System" to your team. You start drawing boxes — "User," "Message," "NotificationService" — and connect them with arrows.

Five minutes later, someone squints and says,

> "Wait… is that arrow showing inheritance or dependency?"

Suddenly, what looked like a clear design becomes a chaotic scribble. Everyone's interpretation is different.

That's where **UML (Unified Modeling Language)** steps in — not as bureaucracy, but as a **shared visual language** for developers. It's how teams at Google, Netflix, and Amazon discuss design before writing a single line of code.

Welcome to **Day 6 of Learn Low-Level Design in 25 Days**, where we explore the **UML basics every engineer should know**. By the end of this post, you'll be able to read and sketch the most useful UML diagrams — the kind that help you explain your system cleanly to teammates, interviewers, and even your future self.



Why UML Exists

Think of UML as **the blueprint of your codebase** — just like an architect uses floor plans before laying bricks. It helps you:

- Visualize relationships between classes, interfaces, and modules
- Communicate design ideas unambiguously
- Catch design flaws early, before they cost you 3 days of refactoring

UML has many diagram types (14 officially!), but in **low-level design**, you'll mostly use these three:

1. **Class Diagram** — shows classes and relationships
2. **Sequence Diagram** — shows how objects interact over time
3. **Use Case Diagram** — shows what users (actors) can do in the system

Let's focus on the first two — they're the real workhorses for engineers.

A. Class Diagrams — The Skeleton of Your System

A **class diagram** answers:

> "What are the main objects, and how are they related?"

Here's a simple example for a **Library Management System**:



Let's decode some UML symbols:

- ◦ → public method or attribute
- ◦ → private
- <> → aggregation (has-a relationship)
- |-- → inheritance

So this diagram says:

- A `Library` has many `Book`s
- A `Book` has one or more `Author`s
- Each class exposes methods (`borrow`, `returnBook`, etc.)

When designing your classes, these diagrams help you think about:

- What attributes belong where
- Which class should handle what logic
- How tightly coupled your design is

B. Sequence Diagrams — The Flow of a Feature

A **sequence diagram** shows how objects talk to each other over time. It's like a timeline of method calls.

Let's say a user borrows a book:



What it tells us:

1. `User` calls `Library.borrowBook()`
2. `Library` fetches a `Book` and calls its `markAsBorrowed()` method
3. The flow returns confirmation to the user

This visual is incredibly useful during design reviews or interviews — it shows not just **what** the system does, but **how** objects collaborate to do it.

UML in a Nutshell

```
+--------------------+
| UML Diagram Types |
+--------------------+
| Class Diagram     | → What classes exist & how they relate  |
| Sequence Diagram  | → How they interact over time           |
| Use Case Diagram  | → What the user can do in the system    |
+--------------------+
```

And relationships cheat sheet:

```
Inheritance      --->  A --> B
Association      --->  A — B
Aggregation      --->  A <>-- B
Composition      --->  A ■-- B
Dependency       --->  A ..> B
```

Try It Yourself

Challenge #1: Sketch a UML **class diagram** for a simple **Food Delivery App**. Hint: You'll probably have classes like `Restaurant`, `MenuItem`, `Order`, and `Customer`.

Challenge #2: Create a **sequence diagram** for "Customer places an order and gets confirmation."

Ask yourself:

- Which object initiates the process?
- Who responds to whom?
- Where should the main business logic live?

Summary + Next-Day Teaser

Key Takeaways

- UML is not documentation overhead — it's your design clarifier.
- Class diagrams show structure; sequence diagrams show behavior.
- Use UML to align your mental model with your team's.
- Start with rough sketches; perfection isn't the goal — clarity is.

When to Use UML:

- During design reviews, interviews, or feature planning.

When to Avoid It:

- For trivial features or when the design is already obvious in code.

Tomorrow, we'll **design a real-world Parking Lot System**, and you'll see how UML diagrams evolve into actual code and classes.

If you're following this series, save the complete list here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)